Field Hours

CSCE 614 Computer Architecture

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 40 minutes

The machine and its bottleneck

A stored-program computer keeps instructions and data in the same memory. The processor fetches an instruction, decodes it, executes it, and writes a result. That loop is the von Neumann design, and the bottleneck is the path to memory: the processor can issue work faster than memory can supply operands. Architecture after that design is a set of attempts to keep the processor busy anyway. Pipelines, caches, vectors, and multiple processors are those attempts. They do not repeal the bottleneck. They hide it.

Performance is instructions times cycles per instruction times the clock period. Change one factor and say which one you changed. A shorter clock does nothing for a program that now takes more cycles because the pipeline stalls. A smaller instruction count does nothing if each instruction waits on memory. Report time, not a clock speed.

Instruction sets are the contract. A reduced set makes the pipeline simpler and shifts complexity into the compiler and the instruction count. A richer set can do more per instruction and can be harder to pipeline. This course does not pick a side as fashion. It asks, for a workload, which term in the product moved.

Dataflow is the contrasting picture. An instruction runs when its operands are ready, not when a program counter arrives. A von Neumann core pretends the program is a line. A dataflow view admits it is a graph. Superscalar hardware is a limited dataflow engine wearing a program-counter costume: it finds ready instructions inside a window and runs them together.

Time is the product of three terms. Name the term a change actually moves.

How many -> Cycles each -> Seconds / cycle -> Seconds

Two billion instructions

1. The program executes 2 billion instructions. CPI is 1.5. The clock is 2 billion cycles per second.

2. Cycles required: 2 billion times 1.5 = 3 billion. Time: 3 billion / 2 billion = 1.5 seconds.

3. A redesign with CPI 1.0 and the same clock takes 1.0 second. Speedup is 1.5, which is the old time over the new time.

Problem 1. The clock doubles and the CPI also doubles, instruction count unchanged. What is the speedup?

Answer. Time is proportional to CPI over clock frequency. Both double, so the ratio is unchanged. Speedup is 1. The machine is not faster.

Problem 2. A program is 50 percent memory waits that a cache removes, and the other 50 percent is unchanged. What speedup does Amdahl's law give?

Answer. The cached fraction takes essentially no time in the idealization where those waits drop to zero. New time is 0.5 of the old time. Speedup is 2. You cannot get more than 2 by perfecting half the program.

Lecture 2 · 45 minutes

Pipelines and hazards

A five-stage pipeline overlaps fetch, decode, execute, memory, and writeback. In the ideal case one instruction finishes every cycle after the pipe fills, so the speedup over a single-cycle design approaches the number of stages. The ideal is a diagram. Hazards are the reasons a real pipe stalls. A structural hazard is two stages wanting the same piece of hardware. A data hazard is an instruction that needs a result still in flight. A control hazard is a branch whose target is not known in time to fetch the next instruction.

Forwarding sends a result from a later stage back to execute so the next instruction does not wait for writeback. It does not fix a load-use hazard. A load produces its value at the end of the memory stage. The very next instruction wants that value at the start of execute. The data is one cycle late even with forwarding, so the pipe inserts one stall. A compiler, or a careful human, puts an independent instruction in that slot.

A branch decided in execute has already fetched two instructions that may be wrong, in a pipe where the branch condition is known that late. Those fetches are bubbles if the branch is taken and the prediction was not. A predicted-not-taken policy is simple and wrong about half the time on loop branches, which are usually taken. A one-bit or two-bit predictor at the branch address is the ordinary repair. The misprediction penalty is the number of stages between fetch and the decision.

Deepening the pipe raises the ideal throughput and raises the misprediction penalty. That trade is the content of the lecture. A twenty-stage pipe that mispredicts often can lose to a shorter pipe. Count stalls before you celebrate the clock.

Five stages. A load followed at once by a use needs a stall even if results are forwarded.

Fetch -> Decode -> Execute -> Memory -> Writeback

A load and the add that needs it

1. Instruction one loads a register from memory. Instruction two adds that register to another.

2. Forwarding can supply an add's result to the next add. It cannot supply the load's result before the memory stage finishes.

3. The schedule inserts one bubble between them. The add executes one cycle later than the ideal back-to-back pair.

Problem 1. A program is 20 percent branches. The predictor is wrong 10 percent of the time. The penalty is 2 stall cycles. How many extra cycles per instruction is that?

Answer. Mispredict rate per instruction is 0.20 times 0.10 = 0.02. Times a penalty of 2 is 0.04 extra cycles per instruction. A base CPI of 1 becomes 1.04 from branches alone, before other stalls.

Problem 2. Two independent adds in a row, with forwarding. How many stalls?

Answer. Zero. The first add's result can be forwarded from execute to the second add. A stall is for a

dependence the forward cannot cover, not for every pair of instructions.

Lecture 3 · 45 minutes

Caches

A cache holds a small copy of memory close to the processor. A hit is served from that copy. A miss goes to the next level and pays. The three numbers that describe a cache are capacity, block size, and associativity. Capacity is how much data. Block size is how many neighboring bytes move together. Associativity is how many places a given block is allowed to sit. Direct-mapped means one place. Fully associative means any place.

An address splits into offset, index, and tag. The offset picks the byte inside the block. The index picks the set. The tag checks that the block in that set is the one you asked for, not another block that happens to share the index. A hit is a tag match and a valid bit. Forget the valid bit and an empty line looks like address zero.

Misses have causes. A compulsory miss is the first touch of a block. A capacity miss is a block that was evicted because the cache cannot hold the working set. A conflict miss is a block evicted because too many live blocks wanted the same set, even though the cache had room elsewhere. Raising associativity attacks conflicts and does not attack capacity. Bigger blocks attack compulsory misses when the program uses neighbors, and they can raise conflict misses when it does not.

Writes are the awkward half. A write-through cache updates memory on every store and is simple. A write-back cache updates memory when the block is evicted and must remember a dirty bit. A write-allocate policy brings the block in on a write miss. A no-allocate policy writes around the cache. The policy is part of the machine, not a detail for later.

Address 44, block size 16 bytes, 4 direct-mapped lines. The fields are this split.

Tag 0

Index 2, so line 2

Offset 12, the byte in the block

Where address 44 sits

1. Block size is 16 bytes, so the offset is 44 mod 16 = 12.

2. The block number is floor of 44 / 16 = 2. Four lines, so the index is 2 mod 4 = 2.

3. The tag is floor of 2 / 4 = 0. A hit on line 2 requires valid and tag 0.

Problem 1. Same cache. Does address 44 and address 108 land in the same line?

Answer. 108 divided by 16 is 6.75, so the block number is 6. Index is 6 mod 4 = 2. Yes, the same line. The tags differ: floor of 6 / 4 is 1, not 0. They conflict.

Problem 2. A loop touches 8-byte records that are 64 bytes apart, on a cache with 16-byte blocks. Why do bigger blocks not help much?

Answer. Each block brings 16 bytes and the program uses 8 of them, then jumps 64 bytes. The extra bytes in the block are not the next record. You pay to move neighbors you do not read. This is not spatial locality.

Lecture 4 · 40 minutes

Virtual memory and the TLB

Virtual memory gives each program a private address space and lets that space be larger than physical memory. The hardware translates a virtual page number to a physical frame on every load and store. The page table, in memory, holds the map. If the translation itself required a memory read every time, you would have doubled the cost of every load. The translation lookaside buffer is a cache of page-table entries, and a TLB hit is what makes the scheme tolerable.

A page fault is a miss in the page table's valid bits: the page is on disk, or it has never been used. The processor traps to the operating system. The operating system picks a frame, reads the page, updates the table, and restarts the instruction. The architectural fact is the trap and the restart. Which page to evict is a policy, and it belongs to the operating system lecture as much as to this one.

Page size is a compromise you can compute. A larger page wastes space when a program uses only part of it, and it shortens the page table and raises the chance a TLB entry covers useful data. A 4-kilobyte page on a 32-bit address uses 12 offset bits and 20 bits of virtual page number. A flat table of that size is large, which is why real machines use multi-level tables. The TLB hides the levels when the working set of pages is small.

Caches can be indexed before translation or after. A virtually indexed cache is fast and must be careful when two virtual addresses in different processes map to the same physical frame, or when the same virtual address means different frames. The bug is called a synonym or a homonym depending on which way the confusion runs. The lecture's demand is that you can say which address, virtual or physical, is in the tag.

A load, when the TLB holds the translation. A miss in the TLB is not yet a page fault.

1. Split the virtual address

2. Look up the TLB

3. On a hit, form the physical address

4. Then consult the cache

Page size 4096, address 20480

1. 4096 times 5 is 20480, so the virtual page number is 5 and the offset is 0.

2. Suppose the page table says page 5 lives in frame 9. The physical address is 9 times 4096, plus 0, which is 36864.

3. A TLB that already maps 5 to 9 does not read the page table. A TLB miss does, and then fills the TLB. The page is still present. That is not a fault.

Problem 1. Address 20481 with the same page size. What are the page number and offset?

Answer. Page 5, offset 1. Only the byte inside the page changed. The translation is the same frame.

Problem 2. A TLB miss occurs and the page-table entry is valid. Is this a page fault?

Answer. No. A page fault is an invalid or not-present translation. A TLB miss with a valid entry is a refill. The program should not enter the fault handler.

Lecture 5 · 40 minutes

Vectors, multiprocessors, and dataflow

A vector or SIMD instruction applies one operation to several lanes of data. It attacks the instruction-count term: one fetch drives many adds. It helps when the lanes do the same thing. A branch that takes different directions in different lanes turns into a mask, and the unused lanes still occupy the hardware. Vector code that is 10 percent taken-branches can run at a small fraction of the peak you quoted from the brochure.

A multiprocessor puts several von Neumann cores next to a memory. If they share that memory, coherence is the problem: two caches can hold the same line, and a write in one must not leave the other holding a stale word. A coherence protocol marks lines modified, shared, or invalid. The protocol is correctness. The performance question is how often two cores write the same line. False sharing is two cores writing different words that happen to sit in one line, so the protocol bounces the line anyway.

A graphics processor pushes the same idea further: many simple lanes, a memory system that wants long sequential transfers, and a programming model that assumes you have thousands of independent elements. It is a bad machine for a chain of dependent scalar decisions. It is a good machine for a map over an array. Calling it a faster CPU is how people write slow programs on expensive hardware.

Dataflow ties the lecture back to the first one. A dependence graph with wide parallel ranks runs well on SIMD and on many cores. A long chain of dependences runs at the speed of one lane, no matter how many you bought. Draw the graph of the loop before you pick the machine. The bottleneck you hide is the one the graph actually has.

Same operation, two machines. The graph of the work decides which one is real.

Wide and independent

Many lanes stay busy

SIMD and GPUs fit

The bottleneck was fetch

A long chain

One lane does the work

Extra lanes wait

The bottleneck is dependence

False sharing on a 64-byte line

1. Core A writes a counter at byte 0. Core B writes a different counter at byte 32.

2. Both bytes live in one cache line. Coherence treats the line as one unit.

3. Each write invalidates the other core's copy. The line crosses the interconnect on every update even though the words are independent.

Problem 1. You pad the two counters into separate 64-byte lines. What changes?

Answer. The writes hit different lines, so coherence traffic for those writes stops. The program did not become more parallel in its real data. You removed a false conflict.

Problem 2. A loop body is x = f(x) with no other inputs, repeated n times. Why does a 32-lane SIMD unit not approach 32 times speedup?

Answer. Each iteration depends on the previous x. The lanes cannot compute step k+1 until step k finishes. The extra lanes have nothing independent to do. The chain is the bottleneck.

Lecture 6 · 180 minutes

Instruction-level parallelism

A pipeline overlaps instructions. Instruction-level parallelism is how many of those instructions are actually independent. A program with a chain of dependent adds has almost none, no matter how wide the machine is. A program with independent adds has as much as the window can see.

A scoreboard issues an instruction when its functional unit is free and its source registers are ready, and it stalls on write conflicts. Tomasulo's algorithm adds reservation stations and register renaming, so a write to a register does not wait for an earlier write to the same name. The name was the conflict. The values were not.

Speculation executes past a branch before the branch resolves, using a prediction. If the prediction is wrong, the machine squashes the speculative results and must not have written them into the architectural state. Reorder buffers exist so that results commit in program order even if they finished out of order.

Width is not throughput. If the branch is mispredicted every five instructions and the penalty is 15 cycles, the machine spends more time refilling than executing. Compute that before you buy a wider core.

Issue, execute, commit.

Rename the destination -> Wait in a reservation station -> Execute when inputs arrive -> Commit in program order

Two adds that share a name

1. Instructions: R1 = R2 + R3, then R1 = R4 + R5, then R6 = R1 + R7. The second write to R1 does not depend on the first. The third instruction depends on the second.

2. Without renaming, the second add waits so it does not overwrite R1 before a hypothetical reader of the first result. There is no such reader here, but the name collides.

3. With renaming, the first R1 and the second R1 are different physical registers. The second add issues immediately. The third waits only for the second.

Problem 1. A chain of 20 dependent integer adds runs on a 4-wide machine. What is the best throughput you should expect, in adds per cycle?

Answer. About one add per cycle, plus pipeline fill, because each add needs the previous result. Width 4 does not create independence. The extra issue slots stay empty.

Problem 2. Speculation writes a store into the data cache before the branch resolves, and the branch was wrong. What must the machine be able to do?

Answer. Undo the store, or not have made it visible yet. A reorder buffer that commits stores only when the branch is known prevents the visible write. A machine that cannot undo or delay the store has corrupted architectural state.

Lecture 7 · 180 minutes

Branches

A conditional branch is a data dependence from the condition to every instruction after it. The pipeline either stalls until the condition is known or guesses. A stall of the full pipeline depth on every branch is intolerable, because branches are common. Guessing is the design.

A bimodal predictor stores a two-bit counter per branch address. Taken and not-taken increment and decrement the counter, and the prediction is the high bit. Two bits tolerate one stray outcome without flipping the prediction. A one-bit counter flips on every loop exit and then mispredicts the next entry.

A branch target buffer caches the destination. Predicting taken is useless if you do not know where. Indirect branches, including returns, need a structure that can hold more than one target. A return-address stack predicts returns by the call nesting, which a flat table does poorly.

Measure the misprediction rate before you optimize a branchy program. If the rate is already under one percent, the branch is not your bottleneck. If a two-bit counter fails on a pattern that alternates in a regular way, a local-history predictor is the next tool, not a wider core.

A two-bit counter saturates.

00 strongly not taken

01 weakly not taken

10 weakly taken

11 strongly taken

A loop that exits

1. A loop is taken 100 times and then not taken once. A one-bit predictor mispredicts the exit and then mispredicts the first taken branch of the next run.

2. A two-bit counter that sat at 11 goes to 10 on the exit and still predicts taken next time. One stray outcome does not flip it.

3. The exit itself is still a misprediction. The counter does not foresee the trip count unless a different predictor stores that history.

Problem 1. Why does a one-bit predictor mispredict twice per loop, while a saturated two-bit counter mispredicts once?

Answer. The exit flips the one-bit predictor to not-taken, so the next iteration's first branch, which is taken, misses as well. The two-bit counter moves from strongly taken to weakly taken and still predicts taken. Only the exit misses.

Problem 2. You correctly predict that a branch is taken and then spend 12 cycles computing its target.

Did the predictor remove the bubble?

Answer. No. The direction was ready and the target was not. You need the target in a buffer, or you still stall. Direction and target are separate predictions.

Lecture 8 · 180 minutes

Coherence

Two cores with private caches can hold two different values for one address if nothing stops them. Coherence is the protocol that stops them. The single-writer, multiple-reader invariant says that at any moment either one core may write, or several may read, but not both. MESI names the states: Modified, Exclusive, Shared, Invalid.

A core that wants to write an address that another core holds must invalidate the other copies first. A core that wants to read an address another core has modified must get the new value and the writer's copy becomes shared or invalid. The bus or the directory is how those messages travel. Without a protocol, the program's stores are suggestions.

False sharing is coherence traffic on a line that holds two unrelated variables. Two cores increment two counters that happen to sit on the same 64-byte line, and every increment invalidates the other core. The program is race-free and still slow. Pad the counters onto separate lines if the profile shows it.

Coherence is not consistency. Coherence talks about one address. Consistency talks about the order of accesses to different addresses as other cores see them. A program can be coherent and still surprise you if you expected sequential consistency and the machine reordered stores.

One line, two questions.

Coherence

One address

MESI states

Invalidations

Consistency

Several addresses

What order another core sees

Fences when the language requires them

False sharing on 64 bytes

1. Counter A and counter B sit 8 bytes apart, on one cache line. Core 0 increments A. Core 1 increments B.

2. Each increment is a write. The protocol grants the line to one writer and invalidates the other. The line bounces.

3. Move B to the next line. The invalidations stop. The increments were never a data race. The line size was the coupling.

Problem 1. Core 0 holds a line in Modified and Core 1 wants to read it. What must happen before Core 1's load returns?

Answer. The modified value must reach Core 1, and Core 0's copy must stop being the unique dirty copy. In MESI, Core 0 supplies the data and both end in Shared, or the writer's line is written back and then shared. Core 1 must not read a stale copy.

Problem 2. You add a lock around both counters and the line still bounces. Why?

Answer. The lock serializes the updates and does not move the counters onto different lines. Coherence traffic follows the writes, even when the writes are ordered. False sharing is a layout problem. The lock answers a different problem, a race.

Lecture 9 · 180 minutes

A memory system you can measure

A cache is defined by capacity, line size, and associativity. The earlier lecture placed one address. This week stacks the misses. A compulsory miss is the first touch of a line. A capacity miss is a line that was evicted because the working set does not fit. A conflict miss is a line evicted because too many hot lines mapped to the same set, even though the cache as a whole had room.

Changing associativity separates conflict from capacity. If a direct-mapped cache misses and a fully associative cache of the same size does not, the misses were conflicts. If both miss, you need a bigger cache or a smaller working set. Buying associativity will not fix a working set that does not fit.

Prefetching hides latency when the next address is predictable, and it pollutes the cache when it is not. A prefetcher that pulls the next line helps a scan and hurts a pointer chase whose next address is a loaded value. Profile the two patterns separately.

Write the miss rate next to the access count. A 1 percent miss rate on a memory-bound kernel can still dominate if the miss penalty is hundreds of cycles. The rate without the penalty is not a time.

Misses, separated.

Compulsory: first touch

Capacity: working set exceeds the cache

Conflict: the set was too small

The penalty in cycles, not just the rate

Where address 44 sits, again, with a neighbor

1. Line size 16 bytes, 4 sets, direct-mapped. Address 44 and address 44 + 64 map to the same set if 64 is a multiple of the set stride. 16 times 4 is 64, so they alias.

2. Touching them alternately on a direct-mapped cache misses every time after the first fill of each, if only one line fits in the set.

3. A 2-way cache of the same capacity holds both. The misses were conflicts. Capacity was not the cause.

Problem 1. The working set is twice the cache and accesses are sequential. Will raising associativity from 1 to 8 remove most misses?

Answer. No. Those are capacity misses. Associativity changes which line you evict inside a set. It does not make the cache larger. You need fewer live lines or a larger cache.

Problem 2. A prefetch of the next line drops misses on a scan and raises them on a hash-table probe. Why is that one mechanism doing both?

Answer. The scan's next address is the next line, so the prefetch is used. The probe's next address is not the next line, so the prefetched line evicts something you needed. The mechanism has no knowledge of which pattern you are in unless you tell it or it detects the stride.

Lecture 10 · 180 minutes

Multiprocessors

A multicore chip is a set of cores, private caches, and a shared last-level cache or memory controller. The earlier coherence lecture is the contract. This week is the cost. A lock that bounces between cores costs the coherence round trip, not the cost of the increment. Spinning on a lock that another core holds burns issue slots and the interconnect.

A scalable lock backs off, or waits in a queue, so that not every core invalidates the same line on every release. Test-and-test-and-set spins on a local read and only tries the atomic write occasionally. It is still unfair and still noisy. A ticket lock or an MCS queue keeps the handoff on a line the next waiter owns.

The interconnect is a bus only while the core count is small. A crossbar or a mesh has a different bandwidth and a different worst-case hop count. An algorithm that assumes a broadcast invalidate is cheap will not survive a directory protocol where the directory must list the sharers.

Parallel speedup is limited by the serial fraction and by the contention you added. Amdahl's law is the serial fraction. The coherence traffic is the term people forget to put in the model. Measure both.

Where time goes when eight cores share one counter.

The increment itself

The atomic instruction

The line bouncing between cores

The serial fraction you did not parallelize

Amdahl with a lock

1. A job is 10 seconds, of which 1 second is serial. Eight cores, perfect scaling of the rest: time is 1 + 9/8 = 2.125 seconds. Speedup is about 4.7, not 8.

2. Add a lock that costs 0.5 seconds of contention you did not have in the serial run. Time becomes 2.625 seconds. Speedup falls further.

3. The model is crude and it stops you from expecting 8. The measurement on the machine is the number you keep. The model is what you compare it to.

Problem 1. Eight cores spin on one test-and-set lock. Why does throughput of the critical section fall as you add cores past a point?

Answer. Each failed atomic write invalidates the line. More cores mean more invalidations per successful acquire. The lock line spends its time moving. A queued lock hands off to one waiter and the others spin locally, or sleep.

Problem 2. The serial fraction is 0 and the speedup from 1 to 8 cores is 2. What does Amdahl's law not explain, and where do you look?

Answer. Amdahl with a zero serial fraction predicts speedup 8. The missing factor is overhead: contention, false sharing, load imbalance, or a bandwidth limit. Profile the lock and the cache line, not the serial section you already measured as empty.

Lecture 11 · 180 minutes

Vectors and SIMT

SIMD applies one instruction to several lanes of data. The hardware is simple and the programming model is strict: lanes that do not agree on control flow are masked off and the instruction still takes time. A vectorized loop with a divergent branch runs both sides, each with some lanes idle.

A GPU's SIMT model looks like many threads and executes as warps of lockstep lanes. Divergence inside a warp is the same masking. Coalesced memory access means consecutive lanes touch consecutive addresses, so the hardware serves them as one transaction. A warp that gathers one address per lane from all over memory issues many transactions and the arithmetic units wait.

Occupancy is how many warps are in flight to hide memory latency. It is limited by registers and shared memory per thread. Using every register to gain a little arithmetic can drop occupancy and lose the latency hiding. The trade is empirical. The model tells you which knob you turned.

Do not move a branchy, pointer-chasing algorithm onto a GPU and expect the peak FLOP number. That number assumes the lanes agree and the accesses coalesce. A matrix multiply can approach it. A tree walk will not.

The same work, two machines.

SIMD loop

Consecutive lanes, consecutive addresses

One vector load

A branch masks lanes

Gather

Each lane holds a pointer

Many transactions

Arithmetic waits on memory

Four lanes, one branch

1. Lanes hold 1, 2, 3, 4. The branch keeps even values.

2. The even path runs with two lanes active. The odd path runs with the other two. Wall time is both paths, not the longer one alone, on a lockstep warp.

3. If you split the data so a warp is all even, the odd path is a different warp and the masking inside the even warp disappears. Layout changed the divergence.

Problem 1. A warp of 32 lanes loads 32 addresses that are 128 bytes apart. How many cache lines do you touch compared with 32 consecutive floats?

Answer. Thirty-two lines if each address falls in a different line, against one or two lines for consecutive floats. The arithmetic intensity collapses. Coalesce, or accept a memory-bound kernel.

Problem 2. You double the registers per thread and occupancy falls by half, and the kernel slows. What was hiding the latency before?

Answer. Other warps. With fewer warps in flight, a memory stall has nothing else to issue. The extra registers may have removed a few instructions and still lost, because the machine was latency-bound, not arithmetic-bound.

Lecture 12 · 180 minutes

Virtual memory at the system level

The earlier lecture computed a page number. This week is the mechanism around it. A page fault on a resident set that is too small thrashes: the pages you need are evicted to bring in the pages you need next, and the disk or the flash becomes the processor. The working set is the pages referenced in a recent window. If it does not fit, add memory or touch less.

A translation lookaside buffer caches page-table entries. A miss costs a walk of the page table, which is itself cached. Huge pages reduce TLB pressure by covering more address with one entry, and they waste memory and increase fragmentation. Use them when the TLB is the measured problem.

Copy-on-write lets a fork share pages until a write. The first write faults, copies the page, and reprotects it. A program that forks and then writes every page pays the copy and gains nothing. A program that forks and execs can share the brief window. The policy matches the use.

The address space is a protection domain. A bit in the page table is the difference between a bug and a stop. Do not turn the bit off to make a debugger convenient and then ship that configuration.

From a virtual address to a stall.

TLB

Page-table walk

Fault if the page is absent

Disk, if the working set does not fit

Page size 4096, address 20480

1. 20480 / 4096 = 5, so this is page 5, offset 0. That part you already did.

2. If the TLB holds page 5, the translation is one cycle away. If not, the walker reads the page-table entry.

3. If the entry is absent because you have 4 frames and you just touched 5 new pages, the fault evicts one. Touch them in a loop and every access faults. That is thrashing, and the page math did not warn you. The working-set size did.

Problem 1. A program sequentially scans an array twice the size of RAM. Will a better replacement policy make the scan run at memory speed?

Answer. No. The working set is the array, and it does not fit. Every policy faults. You need more memory, a smaller scan, or an algorithm that reuses a block before it is evicted.

Problem 2. Fork uses copy-on-write, and the child immediately writes every page. How many page copies do you perform relative to copying everything at the fork?

Answer. The same number, one per written page, plus the fault overhead. Copy-on-write saved nothing on this pattern. It pays when the child writes little, or exits, or replaces its address space.

Lecture 13 · 180 minutes

A performance argument

The last architecture weeks are a method. Pick a kernel you can time. State the peak you could imagine: arithmetic peak or memory peak, whichever the kernel should hit. Measure the kernel. The ratio is the efficiency. Then name the stall that explains the gap, with a counter or a microbenchmark, not with a guess.

A CPI stack splits cycles into retiring instructions, bad speculation, front-end stalls, and back-end stalls. You do not need a vendor tool to start. A back-of-the-envelope roofline compares arithmetic intensity to the machine's balance point. If the kernel does one add per eight bytes from DRAM, it is memory-bound on any machine you can buy, and optimizing the add is wasted time.

Change one thing. Associativity, a padding of one line, a prefetch, a different page size. If you change four things, you do not know which one moved the number.

Write the number with the machine and the input size. A speedup on a 4-kilobyte input that fits in L1 does not transfer to a 4-gigabyte input. The course is the habit of tying the number to the level of the machine that produced it.

The argument, top to bottom.

Peak for this kernel

Measured time

The stall that explains the gap

One change, then measure again

One add per eight bytes

1. A kernel loads two doubles and adds them, streaming. Arithmetic intensity is one add per 16 bytes, or half an add per eight bytes if you count the store. Use the count you can defend.

2. If DRAM delivers 40 GB/s and a double is 8 bytes, the load stream tops out near 5 billion doubles per second, hence that many adds, regardless of a 64-wide vector unit that could do more arithmetic.

3. A vectorized version that does not change the bytes moved will not exceed that ceiling. The stall is memory. The next change is fewer loads, not a wider add.

Problem 1. You optimize the inner arithmetic and the runtime does not change. The roofline says the kernel is under the memory ceiling and far from the arithmetic peak. Where do you look next?

Answer. At the loads and stores: extra copies, a poor layout, or a cache that is not actually holding the reused data. More arithmetic throughput cannot move a kernel that is waiting on bytes.

Problem 2. The kernel is fast on 32 KB and slow on 32 MB, and both sizes are far under RAM. What level of the machine changed?

Answer. The cache. 32 KB may fit in L1 and 32 MB may not fit in the last-level cache, so the slow run is a memory or LLC problem. Report both sizes. One of them is not 'the' runtime.

Lecture 14 · 180 minutes

The machine, end to end

A program runs on a pipeline, fed by a cache, translated by a TLB, on a core that shares lines with other cores. The course split these so each had an invariant. The machine runs them at once. When a number is bad, walk the stack: speculation, then the cache, then the TLB, then coherence, then the memory controller. Stop at the first level that explains the cycles.

You should be able to compute, by hand, a cache index, a page number, a pipeline stall on a load-use, and a speedup bound from a serial fraction. Those four calculations are the literacy. The rest of the course is knowing which one a profile is pointing at.

The hours for this course are those calculations on your own examples, not a reread of these notes. Pick an address and a machine configuration and write the steps. If you cannot finish without looking, you are not done with that week.

Nothing here is a specific vendor's manual. The names MESI, Tomasulo, and TLB are the standard names for the mechanisms. Your measurements belong to the machine you actually timed.

Where to look, in order.

1. Is the pipeline fed?

2. Does the working set fit?

3. Is the TLB the walker?

4. Is a line bouncing?

5. Only then blame the algorithm

One load, four questions

1. A load feeds an add in the next instruction. On a five-stage pipeline with no bypass around the load's memory stage, the add stalls one cycle. That is the pipeline question.

2. The load misses L1 and hits the last-level cache. The stall is tens of cycles, not one. The pipeline diagram was the wrong scale.

3. If the line is in another core's Modified state, add the coherence round trip. Ask the questions in order or you will 'fix' the bypass and change nothing.

Problem 1. Name the calculation that tells you a width-4 core will not speed a dependent chain.

Answer. The length of the dependence chain, in cycles, versus the independent work available in the window. If the chain is longer than the independent work, issue width sits idle. The earlier worked example of twenty dependent adds is this calculation.

Problem 2. You have one afternoon. The program is five times slower than the roofline. What do you measure first?

Answer. Whether it is memory-bound or losing time to mispredicted branches or coherence. A hardware counter or a simple on-and-off of the suspected prefetch or padding tells you which lecture applies. Do not start by rewriting the algorithm.